

Project 2 Foundations: The Resilient Browser (Playwright)

In Project 1, we bypassed the browser completely and talked to the database. But what happens if the data isn't in a clean JSON API? What if the data is protected by CAPTCHAs, encrypted payloads, and complex Javascript that mathematically calculates a secret token before fetching the data?

You can't fake that in Python. You need a real browser to do the math for you.

Enter **Playwright**.

The Core Problem Playwright Solves: JS Rendering

In the old days of scraping, you would use `requests` to download the HTML of a page, and `BeautifulSoup` to find the text.

This no longer works on modern sites (like React or Vue apps). If you use `requests` to download the HTML of a modern website, the HTML file will be almost empty. It will just say: `<div id="root"></div> <script src="app.js"></script>`.

The data isn't in the HTML. The data is inside the Javascript, and the Javascript only executes when it is inside a real browser.

Playwright *is* a real browser (Chromium/Firefox/Webkit). It opens the page, waits for the Javascript to run, waits for the data to be injected into the DOM (the visual layout), and *then* lets you scrape the final, rendered HTML.

The 4 Pillars of Playwright

Before we write code, you must understand these four concepts. If you don't, your scraper will crash randomly (we call this being "flaky").

1. Wait States (The Most Important Concept)

When Playwright clicks a "Load More" button, the data doesn't appear in 1 millisecond. It takes the website time to talk to the server. If your script tries to extract the data immediately after clicking the button, it will extract nothing and crash.

You must explicitly tell Playwright what to wait for.

- **The Bad Way:** `time.sleep(5)` (The site might load in 1 second, meaning you wasted 4 seconds. Or it might take 6 seconds, meaning your script crashes).
- **The Good Way (Network State):** `page.wait_for_load_state("networkidle")` (Wait until the page stops making background network requests).
- **The Best Way (DOM State):** `page.wait_for_selector(".product-card")` (Wait explicitly until the HTML tag for the product card appears on the screen).

2. Headless vs. Headful

- **Headful Mode:** Playwright actually opens a visible Chrome window on your screen. You can sit with a cup of coffee and watch the "ghost" click things and type text. This is mandatory for debugging.
- **Headless Mode:** Playwright runs Chrome invisibly in the background. It uses 1/10th the RAM and CPU. This is what you use when your script is finished and running in production on a server.

3. Locators (How to Point at Things)

To tell Playwright to click a button, you have to tell it *which* button. You do this using Locators (CSS Selectors or XPath).

- `page.locator("button#submit-login").click()` (Find the button with the ID 'submit-login' and click it).
- `page.get_by_text("Load More").click()` (Find any element that has the exact text "Load More" and click it).

4. Browser vs. Context vs. Page

Playwright doesn't just open "a browser". It has a strict hierarchy, designed for speed and scale:

1. **The Browser:** The actual Chromium engine. This is very "heavy" and takes time to launch. You only launch this once.
2. **The Context:** Think of this as an "Incognito Profile". It holds its own cookies and cache. It is very "lightweight" to create. If you want to scrape with 5 different logged-in users simultaneously, you open 1 Browser, but create 5 Contexts.
3. **The Page:** A single tab inside a Context.

Action Plan for Project 2

1. **Target Selection:** We need a site that is difficult. A site where the data requires clicking tabs, expanding dropdowns, or where the HTML is heavily rendered by Javascript.

2. **The Playwright Inspector:** We will use a built-in tool that allows you to click around the website while Playwright *automatically writes the Python code for you*.
3. **Data Extraction:** Once we get the page into the physical state we want, we will rip the HTML and parse it.